

Programming in Java (BCAC 0012)

Module-3

Section – 1

Exception Handling

Faculty Name – Dr. Vinod Jain

Syllabus

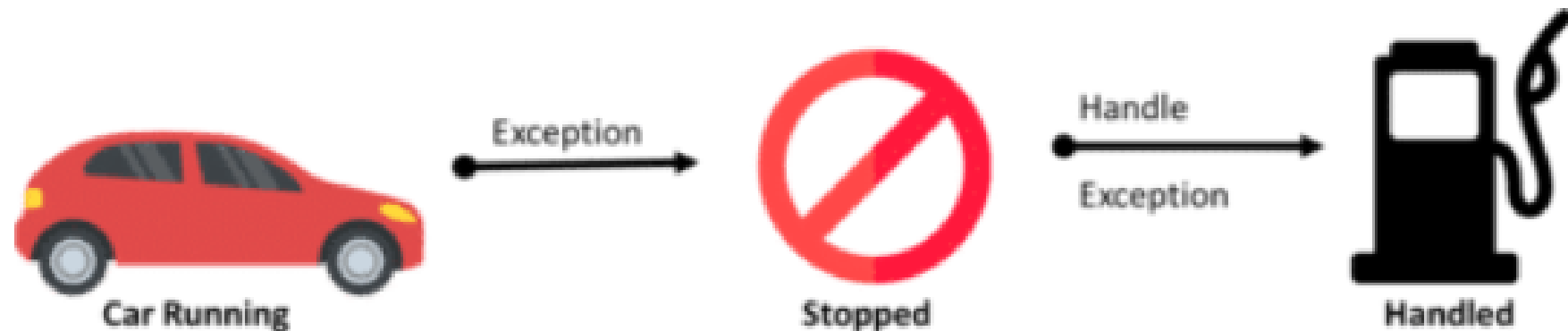
3	Exception Handling	Introduction to Exception, Type of Exceptions
		Keywords-Try ,catch, Throw, Throws, Finally
		Try-catch Block ,Multiple Catch Clauses, Nested Try Statements
		Java's Built-in Exceptions
		Creating Your Own Exception Subclasses.
	Multithreaded Programming	Thread Life Cycle
		Creating Threads , Thread Methods
		Thread Priority,
	Introduction to Applet Programming	Introduction to Applet Programming, Difference between Applet and application
		Graphics class methods and implementation
		Applet Life Cycle and its methods
		Applet tag
		Use of Param tag, Applet Programming

Exception Handling : Syllabus

1. Introduction to Exception
2. Type of Exceptions
3. Keywords-Try, catch,
4. Throw, Throws, Finally
5. Try-catch Block ,
6. Multiple Catch Clauses,
7. Nested Try Statements
8. Java's Built-in Exceptions
9. Creating Your Own Exception Subclasses.

Introduction to Exception

This topic introduces the basic concept of Exception Handling.
So, what if you are driving a car and suddenly something happens and the car stopped. How will you handle that situation?
Say the reason is “No Fuel”.



Introduction to Exception

- An exception (or exceptional event) is a problem that arises during the execution of a program.
- When an Exception occurs the normal flow of the program is disrupted and the program/Application terminates abnormally.
- It is not recommended, therefore, these exceptions are to be handled.

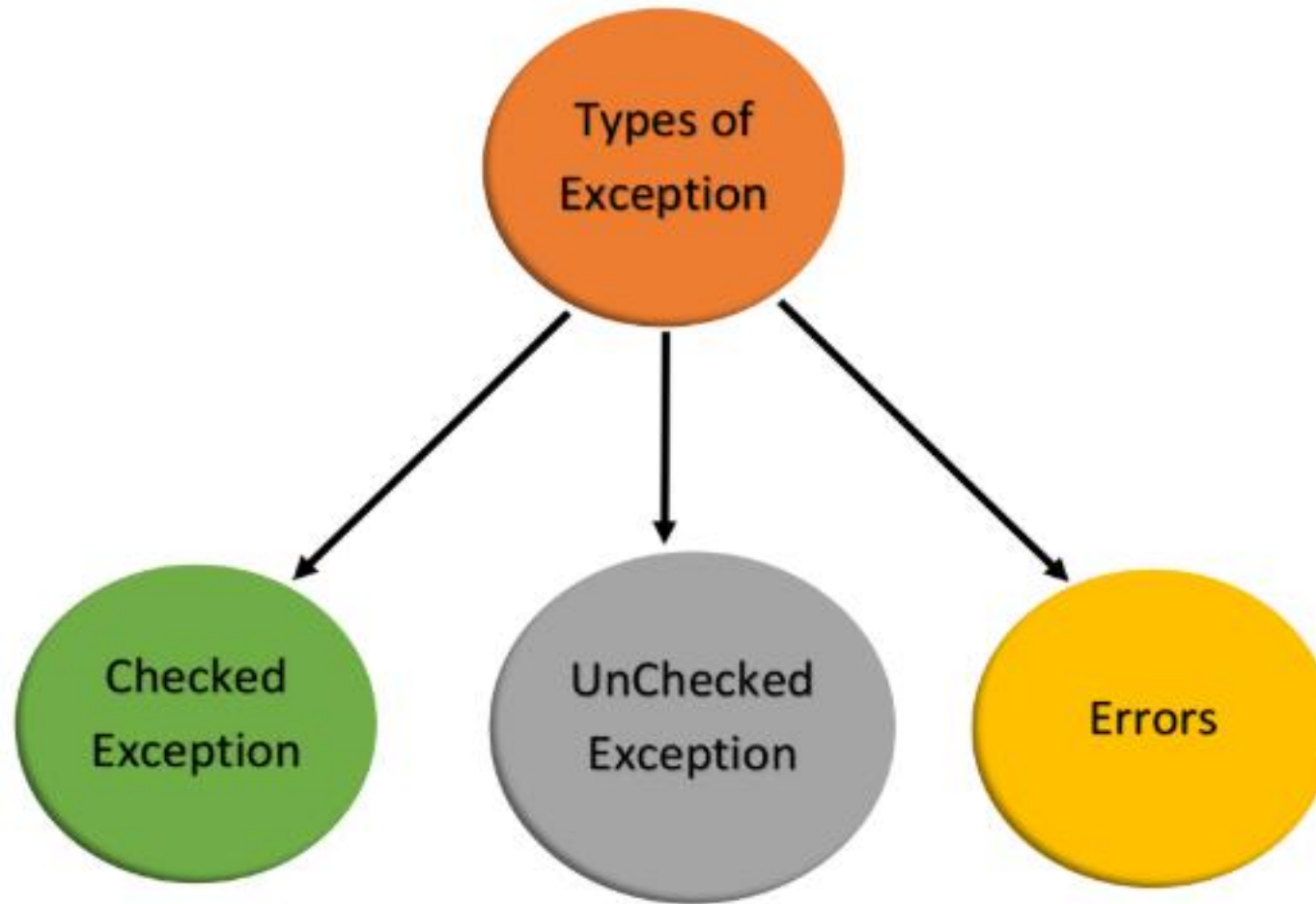
Introduction to Exception

- An exception can occur for many different reasons. Following are some scenarios where an exception occurs.
 1. A user has entered an invalid data.
 2. A file that needs to be opened cannot be found.
 3. A network connection has been lost in the middle of communications or the JVM has run out of memory.
 4. Run time stack overflows
 5. Array index out of exception
 6. Many more

Type of Exceptions

- Exceptions are caused by user error, or by programmer error, or by physical resources that have failed in some manner.
- Based on these, we have three categories of Exceptions.
 1. Checked exceptions
 2. Unchecked exceptions
 3. Errors

Type of Exceptions



Checked exceptions

- A checked exception is an exception that is checked (notified) by the compiler at compilation-time these are also called as compile time exceptions.
- These exceptions cannot simply be ignored, the programmer should take care of (handle) these exceptions.

Checked exceptions

- For example,
- if you use **FileReader** class in your program to read data from a file, if the file specified in its constructor doesn't exist, then a *FileNotFoundException* occurs, and the compiler prompts the programmer to handle the exception.

Checked exceptions

```
public class FileNotFoundException_Demo  
{  
    public static void main(String args[])  
    {  
        File file = new File("E://file.txt");  
        FileReader fr = new FileReader(file);  
    }  
}
```

Checked exceptions

If you try to compile the above program, you will get the following exceptions.

```
C:\>javac FileNotFoundException_Demo.java
```

```
FileNotFoundException_Demo.java:8: error: unreported exception
```

```
FileNotFoundException; must be caught or declared to be thrown
```

```
    FileReader fr = new FileReader(file);
```

```
1 error
```

Unchecked exceptions

- **Unchecked exceptions –**
- An unchecked exception is an exception that occurs at the time of execution.
- These are also called as Runtime Exceptions.
- These include programming bugs, such as logic errors or improper use of an API.
- Runtime exceptions are ignored at the time of compilation.

Unchecked exceptions

- For example,
- if you have declared an array of size 4 in your program, and trying to call the 5th element of the array then an **ArrayIndexOutOfBoundsException** occurs.

Example

```
public class Unchecked_Demo {  
    public static void main(String args[]) {  
        int num[] = {1, 2, 3, 4};  
        System.out.println(num[5]);  
    }  
}
```

Unchecked exceptions

- If you compile and execute the above program, you will get the following exception.
- Output
- Exception in thread "main"
java.lang.ArrayIndexOutOfBoundsException: 5
- at Exceptions.Unchecked_Demo.main(Unchecked_Demo.java:8)

Errors

- These are not exceptions at all, but problems that arise beyond the control of the user or the programmer.
- Errors are typically ignored in your code because you can rarely do anything about an error.
- For example, if a stack overflow occurs, an error will arise.
- They are also ignored at the time of compilation.

Exception Handling (Keywords-Try, catch)

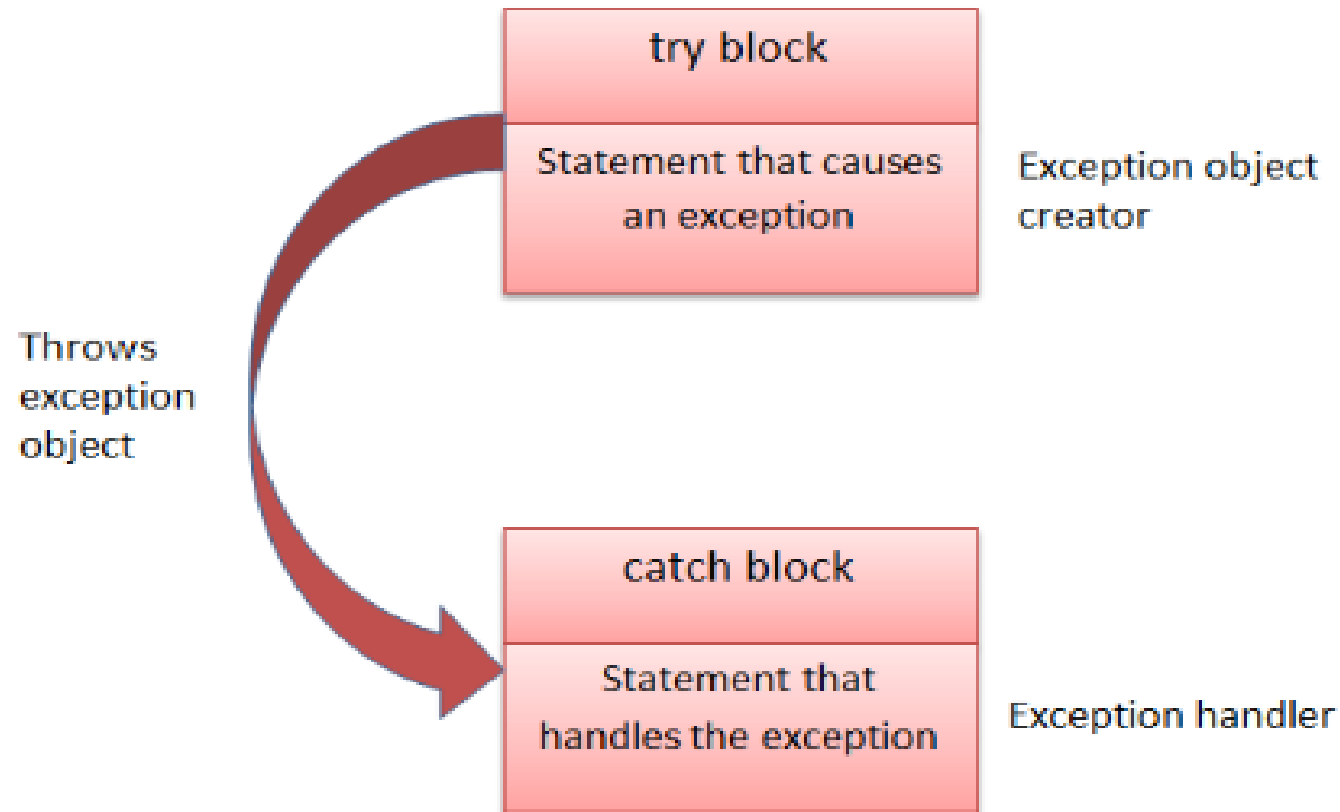


Fig: Exception Handling Mechanism

Exception Handling (Keywords-Try, catch)

- A try/catch block is placed around the code that might generate an exception.
- Code within a try block is referred to as protected code
- The syntax for using try/catch looks like the following –

```
try {  
    // Protected code  
} catch (ExceptionName e1) {  
    // Catch block  
}
```

Exception Handling (Keywords-Try, catch)

- **Try Keyword :**
- The code which is prone to exceptions is placed in the try block. When an exception occurs, that exception occurred is handled by catch block associated with it.
- Every try block should be immediately followed either by a catch block or finally block.

Exception Handling (Keywords-Try, catch)

- **Catch Keyword :**
- A catch statement involves declaring the type of exception you are trying to catch.
- If an exception occurs in protected code, the catch block (or blocks) that follows the try is checked.
- If the type of exception that occurred is listed in a catch block, the exception is passed to the catch block much as an argument is passed into a method parameter.
- The statements in the catch block are used to handle that exception.

Example

```
import java.io.*;

public class ExcepTest {
    public static void main(String args[]) {
        try {
            int a[] = new int[2];
            System.out.println("Access element three :" + a[3]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Exception thrown :" + e);
        }
        System.out.println("Out of the block");
    }
}
```

Output

Exception thrown

:java.lang.ArrayIndexOutOfBoundsException: 3

Out of the block

Another Example

```
public class JavaExceptionExample{  
    public static void main(String args[]){  
        try{  
            //code that may raise exception  
            int data=100/0;  
        }catch(ArithmeticException e){System.out.println(e);}  
        //rest code of the program  
        System.out.println("rest of the code...");  
    }  
}
```

Output

Exception in thread main
java.lang.ArithmeticException:/
by zero
rest of the code...

```

import java.util.Scanner;
public class ExceptionHandling1
{
    public static void main(String args[])
    {
        Scanner sc = new Scanner(System.in);
        int a,b,result;
        try
        {
            System.out.print("\n Enter a : ");
            a = sc.nextInt();
            System.out.print("\n Enter b : ");
            b = sc.nextInt();
            result = a/b;
            System.out.print("\n a/b = " + result);
        }
        catch(Exception e)
        {
            System.out.println("\n Exception occured");
            System.out.println(e.toString());
        }
        System.out.print("\n\n\n");
    }
}

```

run:

Enter a : 10

Enter b : 0

Exception occured
java.lang.ArithmeticException: / by zero

BUILD SUCCESSFUL (total time: 3 seconds)

run:

Enter a : 10

Enter b : 5

a/b = 2

BUILD SUCCESSFUL (to

```

public class ExceptionHandling2 {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        int i,n,sum=0;
        int a[] = new int[3];
        try
        {
            System.out.print("\n Enter n : ");
            n = sc.nextInt();
            for(i=0;i<n;i++)
            {
                System.out.print("\n Enter element : ");
                a[i] = sc.nextInt();
                sum=sum + a[i];
            }
            System.out.print("\n sum = " + sum);
        }
        catch(Exception e)
        {
            System.out.println("\n Exception occured");
            System.out.println(e.toString());
        }
        System.out.print("\n\n\n");
    }
}

```

run:

Enter n : 2

Enter element : 11

Enter element : 112

sum = 123

BUILD SUCCESSFUL (total t

run:

Enter n : 4

Enter element : 5

Enter element : 55

Enter element : 5

Enter element : 553

Exception occured
java.lang.ArrayIndexOutOfBoundsException

BUILD SUCCESSFUL (total ti

Day 2

Topics Covered

- Multiple Catch Clauses
- Throw, Throws, Finally

Multiple Catch Clauses

- The syntax for using try/catch looks like the following –

```
try {  
    // Protected code  
} catch (ExceptionName e1) {  
    // Catch block  
}
```

- The protected code written in try block may throw more than one type of exception.

Multiple Catch Clauses

- For example in the following statements
- $A[i] = a[i+1] / a[i+2]$
- Two exceptions may occur
 1. `ArrayIndexOutOfBoundsException` and
 2. `ArithmeticException` : / by zero
- So a try block can be followed by multiple catch blocks to handle more than one type of exceptions separately.

Multiple Catch Clauses

```
try
{
    // Protected code
}
catch (ExceptionType1 e1) {
}
catch (ExceptionType2 e2) {
}
catch (ExceptionType3 e3) {
}
....
```

Multiple Catch Clauses

- If an exception occurs in the protected code, the exception is thrown to the first catch block in the list.
- If the data type of the exception thrown matches `ExceptionType1`, it gets caught there and first catch block is executed.
- If not, the exception passes down to the second catch statement. This continues until the exception either is caught or falls through all catches.

```
public static void main(String args[])
{
    Scanner sc = new Scanner(System.in);
    int a,b,result;
    try
    {
        System.out.print("\n Enter a : ");
        a = Integer.parseInt(sc.nextLine());
        System.out.print("\n Enter b : ");
        b = Integer.parseInt(sc.nextLine());
        result = a/b;
        System.out.print("\n a/b = " + result);
    }
    catch(ArithmeticException e)
    {
        System.out.println("\n Exception occurred");
        System.out.println(e.toString());
    }
    catch (NumberFormatException ex)
    {
        System.out.println("Number Format Exception " + ex);
    }
}
```

```

public static void main(String args[])
{
    Scanner sc = new Scanner(System.in);
    int a,b,result;
    try
    {
        System.out.print("\n Enter a : ");
        a = Integer.parseInt(sc.nextLine());
        System.out.print("\n Enter b : ");
        b = Integer.parseInt(sc.nextLine());
        result = a/b;
        System.out.print("\n a/b = " + result);
    }
    catch(ArithmeticException e)
    {
        System.out.println("\n Exception occured");
        System.out.println(e.toString());
    }
    catch (NumberFormatException ex)
    {
        System.out.println("Number Format Exception " + ex);
    }
}

```

run:

Enter a : 10

Enter b : 0

Exception occured
java.lang.ArithmeticException: / by zero

BUILD SUCCESSFUL (total time: 6 seconds)

Output - JavaSection2CModule3 (run) X | ExceptionHandling4.java X | ExceptionHandling3.java X | ExceptionHand

run:

Enter a : 10

Enter b : ggg

Number Format Exception java.lang.NumberFormatException:

BUILD SUCCESSFUL (total time: 4 seconds)

Multiple Catch Clauses : code

```
// Multiple Catch
package Help;
import java.util.Scanner;
import java.io.*;
public class ExceptionHandling4
{
    public static void main(String args[])
    {
        Scanner sc = new Scanner(System.in);
        int a,b,result=0;
        try
        {
            System.out.print("\n Enter a : ");
            a = Integer.parseInt(sc.nextLine());
            System.out.print("\n Enter b : ");
            b = Integer.parseInt(sc.nextLine());
```

```
            result = a/b;
            System.out.print("\n a/b = " + result);
        }
        catch(ArithmeticException e)
        {
            System.out.println("\n Exception occurred");
            System.out.println(e.toString());
        }
        catch (NumberFormatException ex)
        {
            System.out.println("Number Format Exception " + ex);
        }
        System.out.print("\n");
    }
}
```

4.0 Throw, Throws, Finally

Throw Keyword

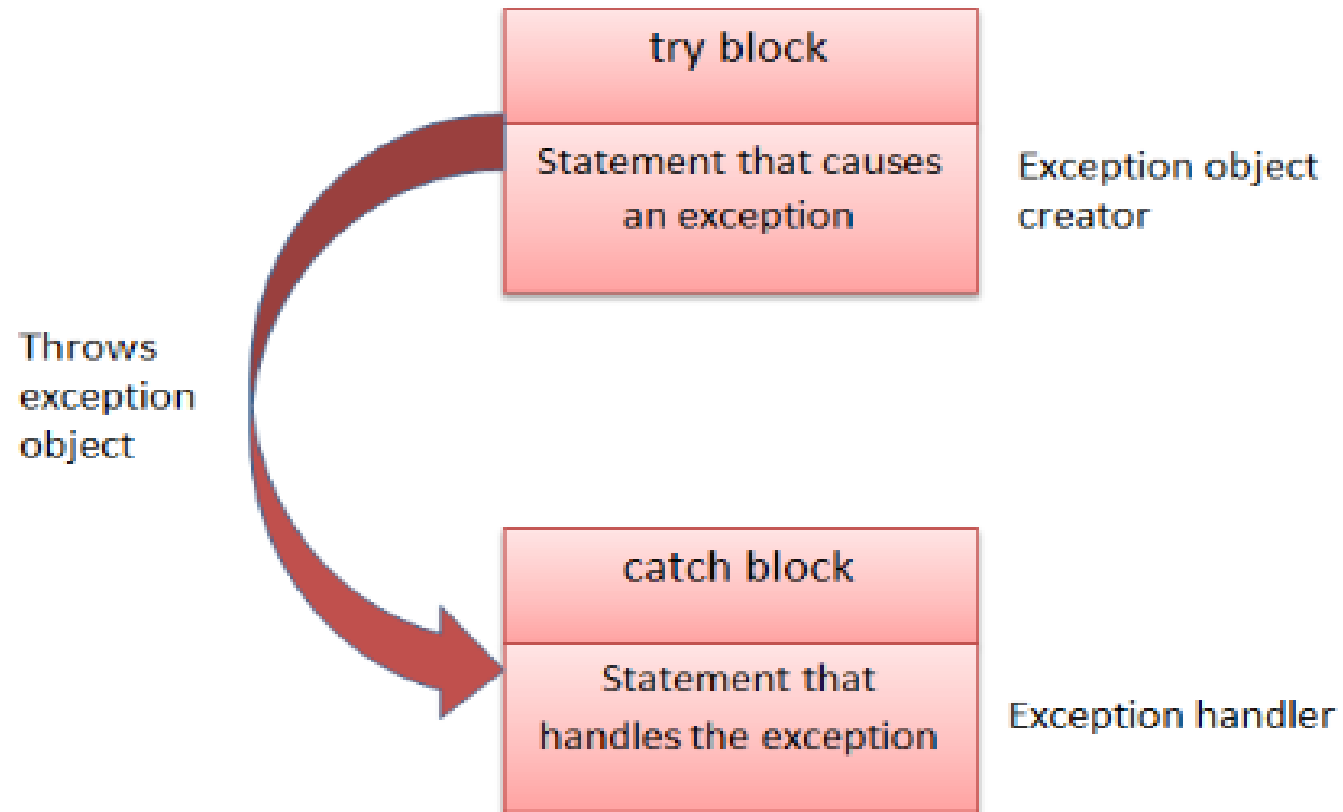


Fig: Exception Handling Mechanism

Throw Keyword

- The Java throw keyword is used to **explicitly throw an exception**.
- The syntax of java throw keyword is given below.
`throw exception;`
`or throw new ArithmeticException("not valid");`
- The throw keyword is mainly used to throw custom exception (own exceptions).
- We will see custom exceptions later in topic Creating Your Own Exception Subclasses.

Throws Keyword

- It is used to declare that a method may throw an exception to its calling method.
- If a method does not handle a checked exception, the method must declare it using the **throws** keyword.
- The throws keyword appears at the end of a method's signature.
- Syntax of java throws

```
return_type method_name() throws exception_class_name  
{  
    //method code  
}
```

Throws Keyword

Example

```
static void validate(int age) throws ArithmeticException  
{  
    if(age<18)  
        throw new ArithmeticException("Age is not valid for voting");  
}
```

```
public class ExceptionHandling6 {  
    static void validate(int age)throws ArithmeticException  
    {  
        if(age<18)  
            throw new ArithmeticException("Age is not valid for voting");  
    }  
    public static void main(String args[])  
    {  
        Scanner sc = new Scanner(System.in);  
        int age;  
        try  
        {  
            System.out.print("\n Enter Your Age : ");  
            age = sc.nextInt();  
            validate(age);  
            System.out.println("\n Welcome to vote");  
        }  
        catch(ArithmeticException ex)  
        {  
            System.out.println("Exception Caught" + ex);  
        }  
    }  
}
```

```

public class ExceptionHandling6 {
    static void validate(int age)throws ArithmeticException
    {
        if(age<18)
            throw new ArithmeticException("Age is not valid for voting");
    }
    public static void main(String args[])
    {
        Scanner sc = new Scanner(System.in);
        int age;
        try
        {
            System.out.print("\n Enter Your Age : ");
            age = sc.nextInt();
            validate(age);
            System.out.println("\n Welcome to vote");
        }
        catch(ArithmeticException ex)
        {
            System.out.println("Exception Caught" + ex);
        }
    }
}

```

run:

Enter Your Age : 20

Welcome to vote

BUILD SUCCESSFUL (total time: 3 seconds)

|

run:

Enter Your Age : 15

Exception Caughtjava.lang.ArithmeticExce

BUILD SUCCESSFUL (total time: 2 seconds)

|

Throws Keyword Code

- // Throw/Throws keywords
- package Help;
- import java.util.Scanner;
- public class ExceptionHandling6 {
- static void validate(int age)throws ArithmeticException
- {
- if(age<18)
- throw new ArithmeticException("Age is not valid for voting");
- }
- public static void main(String args[])
- {
- Scanner sc = new Scanner(System.in);
- int age;
- try

- {
- System.out.print("\n Enter Your Age : ");
- age = sc.nextInt();
- validate(age);
- System.out.println("\n Welcome to vote");
- }
- catch(ArithmeticException ex)
- {
- System.out.println("Exception Caught" + ex);
- }
- }
- }

Throws Keyword

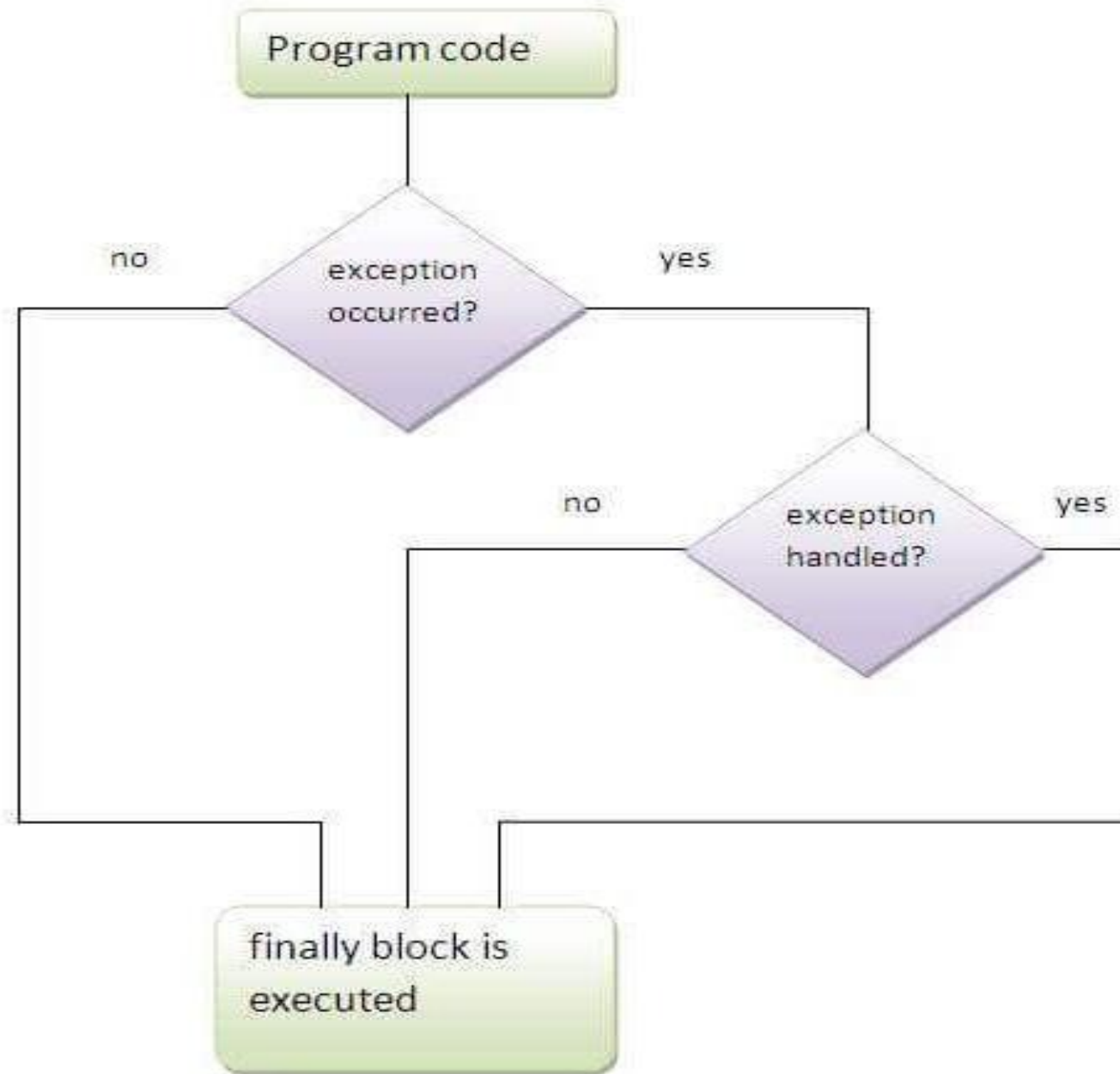
- **Advantage of Java throws keyword**

It provides information to the caller of the method about the exception.

- **Difference between throws and throw keywords**

Throws is used to postpone the handling of a checked exception (to the calling method) and *throw* is used to invoke (and throw an object) an exception explicitly.

Finally



Finally

- A finally block of code always executes, irrespective of occurrence of an Exception.
- The finally block follows a try block or a catch block.
- Using a finally block allows you to run any cleanup-type statements that you want to execute, no matter what happens in the protected code.
- A finally block appears at the end of the catch blocks.

Finally

Syntax

```
try {  
    // Protected code  
} catch (ExceptionType1 e1) {  
    // Catch block  
} catch (ExceptionType2 e2) {  
    // Catch block  
} catch (ExceptionType3 e3) {  
    // Catch block  
}finally {  
    // The finally block always executes.  
}
```

```
public static void main(String args[])
{
    Scanner sc = new Scanner(System.in);
    int a,b,result;
    try
    {
        System.out.print("\n Enter a and b : ");
        a = Integer.parseInt(sc.nextLine());
        b = Integer.parseInt(sc.nextLine());
        result = a/b;
        System.out.print("\n a/b = " + result);
    }
    catch (ArithmeticException ex)
    {
        System.out.println("Arithmetic Exception" + ex);
    }
    catch (NumberFormatException ex)
    {
        System.out.println("Number Format Exception " + ex);
    }
    finally
    {
        System.out.print("\n\n finally block is always executed");
    }
}
```

```

public static void main(String args[])
{
    Scanner sc = new Scanner(System.in);
    int a,b,result;
    try
    {
        System.out.print("\n Enter a and b : ");
        a = Integer.parseInt(sc.nextLine());
        b = Integer.parseInt(sc.nextLine());
        result = a/b;
        System.out.print("\n a/b = " + result);
    }
    catch (ArithmeticException ex)
    {
        System.out.println("Arithmetic Exception" + ex);
    }
    catch (NumberFormatException ex)
    {
        System.out.println("Number Format Exception " + ex);
    }
    finally
    {
        System.out.print("\n\n finally block is always executed");
    }
}

```

run:

Enter a and b : 10

0

Arithmetic Exceptionjava.lang.Arit

finally block is always executed

BUILD SUCCESSFUL (total time: 4 se

Output - JavaSection2CModule3 (run) X ExceptionHandling4.java X

run:

Enter a and b : 10

5

a/b = 2

finally block is always executed

BUILD SUCCESSFUL (total time: 2 se

Day 3

6.0 Nested Try Statements

- The try block within a try block is known as nested try block in java.
- Sometimes a situation may arise where a part of a block may cause one error and the entire block itself may cause another error.
- In such cases, exception handlers have to be nested.

6.0 Nested Try Statements : Syntax

```
try
{
    statement 1;
    statement 2;
    try
    {
        statement 1;
        statement 2;
    }
    catch(Exception e)
    {
    }
}
catch(Exception e)
{
} |
```

// Nested Try Blocks

package Help;

import java.util.Scanner;

public class ExceptionHandling8 {

public static void main(String args[]) {

Scanner sc = new Scanner(System.in);

int a=0,b=0,result=0;

try {

System.out.print("\n Enter a and b : ");

a = Integer.parseInt(sc.nextLine());

b = Integer.parseInt(sc.nextLine());

try {

result = a/b;

System.out.print("\n a/b = " + result);

}

catch (ArithmeticException ex) {

System.out.println("Arithmetic Exception" + ex);

}

}

catch (NumberFormatException ex) {

System.out.println("Number Format Exception " + ex);

}

System.out.print("\n");

}

}

```
// Nested Try Blocks
```

```
package Help;
```

```
import java.util.Scanner;
```

```
public class ExceptionHandling8{
```

```
    public static void main(String args[]) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        int a=0,b=0,result=0;
```

```
        try {
```

```
            System.out.print("\n Enter a and b : ");
```

```
            a = Integer.parseInt(sc.nextLine());
```

```
            b = Integer.parseInt(sc.nextLine());
```

```
            try{
```

```
                result = a/b;
```

```
            System.out.print("\n a/b = " + result);
```

```
            }
```

```
            catch (ArithmeticException ex) {
```

```
                System.out.println("Arithmetic Exception" + ex);
```

```
            }
```

```
        }
```

```
        catch (NumberFormatException ex) {
```

```
            System.out.println("Number Format Exception " + ex);
```

```
        }
```

```
        System.out.print("\n");
```

```
    }
```

```
}
```

Output 1

run:

Enter a and b : 10

tt

Number Format Exception java.lang.NumberFormatException:

BUILD SUCCESSFUL (total time: 3 seconds)

Output 2

run:

Enter a and b : 10

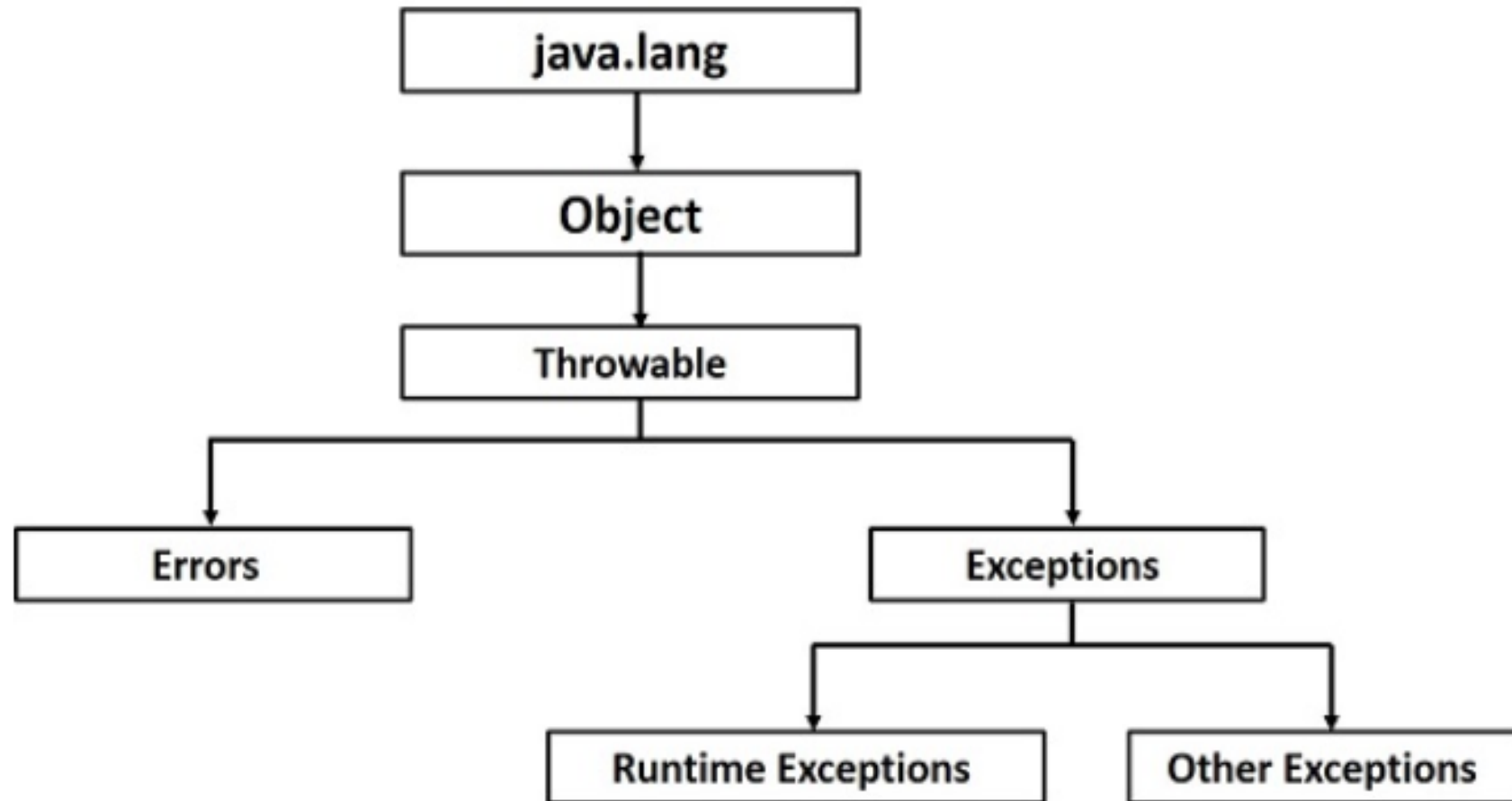
0

Arithmetic Exception java.lang.ArithmeticException: / by zero

BUILD SUCCESSFUL (total time: 2 seconds)

7.0 Java's Built-in Exceptions

7.0 Java's Built-in Exceptions



7.0 Java's Built-in Exceptions

- Java is having a rich set of built in exception classes.
- It includes classes of all types of major exceptions that may occur. Eg. `ArithmeticException`, `ArrayIndexOutOfBoundsException` etc.

7.0 Java's Built-in Exceptions

- All exception classes are subtypes of the `java.lang.Exception` class.
- The exception class is a subclass of the `Throwable` class.
- Other than the exception class there is another subclass called `Error` which is derived from the `Throwable` class.

7.0 Java's Built-in Exceptions

- Errors are abnormal conditions that happen in case of severe failures, these are not handled by the Java programs.
- Errors are generated to indicate errors generated by the runtime environment.
- Example: JVM is out of memory. Normally, programs cannot recover from errors.

7.0 Java's Built-in Exceptions

- The figure in the next slide shows all important exception classes declared in java.lang package.

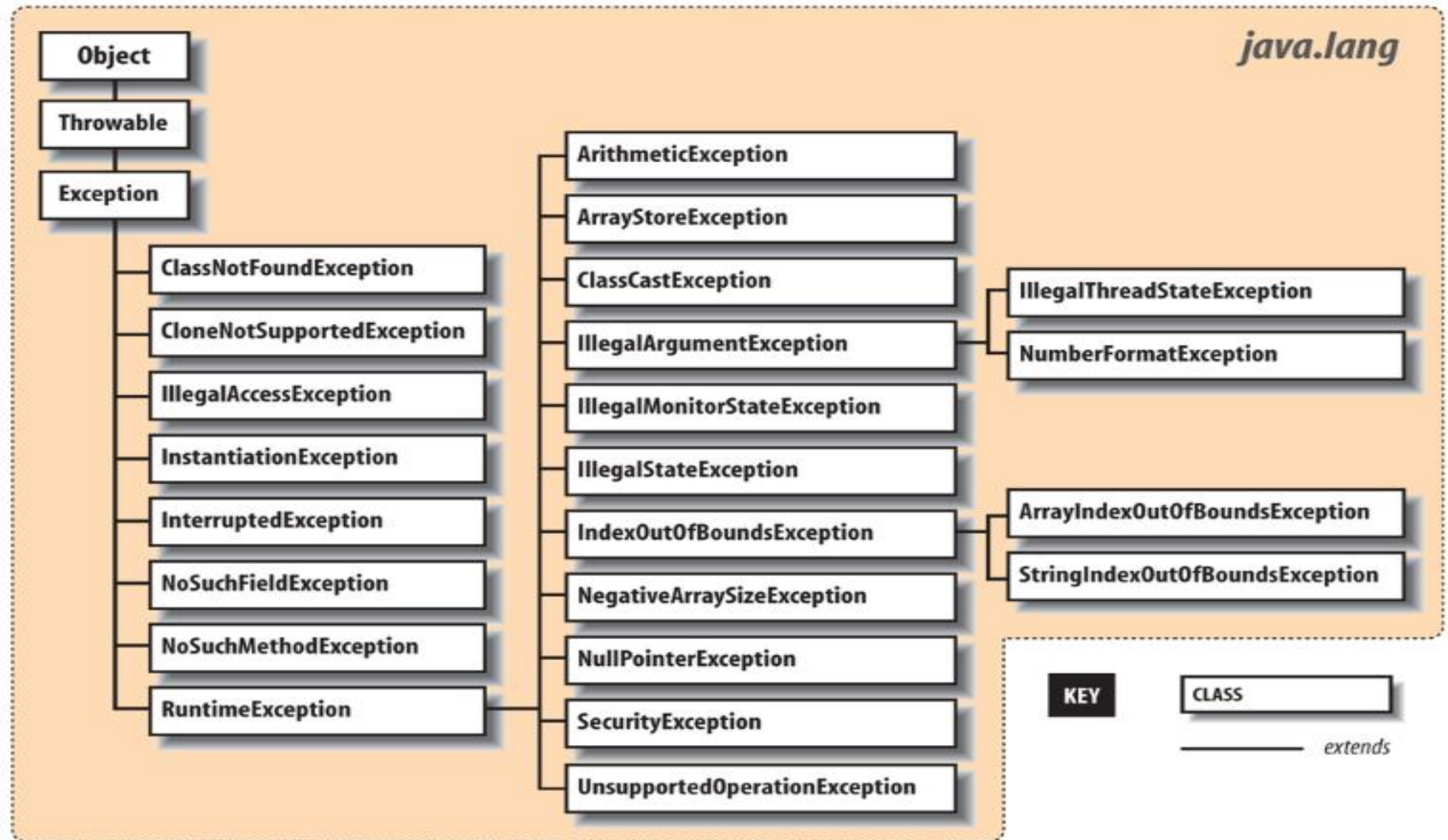


Figure 4-1. The *java.lang.Exception* subclasses

7.0 Java's Built-in Exceptions

- Following is the list of Java Checked Exceptions Defined in java.lang.

Sr.N o.	Exception & Description
1	ClassNotFoundException Class not found.
2	CloneNotSupportedException Attempt to clone an object that does not implement the Cloneable interface.
3	IllegalAccessException Access to a class is denied.
4	InstantiationException Attempt to create an object of an abstract class or interface.
5	InterruptedException One thread has been interrupted by another thread.
6	NoSuchFieldException A requested field does not exist.
7	NoSuchMethodException A requested method does not exist.

7.0 Java's Built-in Exceptions

- Following is the list of some Java Unchecked RuntimeException.

Sr.No.	Exception & Description
1	ArithmeticException Arithmetic error, such as divide-by-zero.
2	ArrayIndexOutOfBoundsException Array index is out-of-bounds.
3	ArrayStoreException Assignment to an array element of an incompatible type.
4	ClassCastException Invalid cast.
5	IllegalArgumentException Illegal argument used to invoke a method.
6	IllegalMonitorStateException Illegal monitor operation, such as waiting on an unlocked thread.
7	IllegalStateException Environment or application is in incorrect state.
8	IllegalThreadStateException Requested operation not compatible with the current thread state.
9	IndexOutOfBoundsException Some type of index is out-of-bounds.
	And many more....

8.0 Creating Your Own Exception Subclasses.

- If you are creating your own Exception that is known as custom exception or user-defined exception.
- You can create your own exception classes by inheriting the Exception class.
- Java custom exceptions are used to customize the exception according to user need.
- By the help of custom exception, you can have your own exception and message.
- Let's see a simple example of java custom exception.

8.0 Creating Your Own Exception Subclasses.

Syntax

```
class MyException extends Exception
{

}
```

Here a new exception class called MyException is crteated

8.0 Creating Your Own Exception Subclasses.

Syntax

```
class MyException extends Exception  
{  
}
```

Example

```
class InvalidAgeException extends Exception  
{  
    InvalidAgeException(String s){  
        super(s);  
    }  
}
```

```
// Creating Your Own Exception Subclasses
```

```
import java.util.Scanner;
```

```
class InvalidAgeException extends Exception
```

```
{  
    InvalidAgeException(String s){
```

```
        super(s);
```

```
    } |
```

```
}
```

```
class TestCustomException1 {
```

```
    static void validate(int age)throws InvalidAgeException{
```

```
        if(age<18)
```

```
            throw new InvalidAgeException("Age is not valid for voting");
```

```
        else
```

```
            System.out.println("Eelcome to vote");
```

```
    }
```

```
    public static void main(String args[]){
```

```
        Scanner sc = new Scanner(System.in);
```

```
        try {
```

```
            System.out.print("\n Enter age : ");
```

```
            int age = Integer.parseInt(sc.nextLine());
```

```
            validate(age);
```

```
        }catch(Exception m){System.out.println("Exception occurred: "+m);} 
```

```
        System.out.println("rest of the code...");
```

```
    }
```

```
}
```

```
// Creating Your Own Exception Subclasses
import java.util.Scanner;
class InvalidAgeException extends Exception
{
    InvalidAgeException(String s){
        super(s);
    }
}
class TestCustomException1 {
    static void validate(int age)throws InvalidAgeException{
        if(age<18)
            throw new InvalidAgeException("Age is not valid for voting");
        else
            System.out.println("Eelcome to vote");
    }
    public static void main(String args[]){
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("\n Enter age : ");
            int age = Integer.parseInt(sc.nextLine());
            validate(age);
        }catch(Exception m){System.out.println("Exception occured: "+m);}
        System.out.println("rest of the code...");
    }
}
```

Output 1

```
run:
    Enter age : 15
Exception occured: InvalidAgeException: Age is not valid for voting
rest of the code...
BUILD SUCCESSFUL (total time: 2 seconds)
```

Output 2

```
run:
    Enter age : 25
Eelcome to vote
rest of the code...
BUILD SUCCESSFUL (total time: 2 seconds)
```